Task 4: Gamma (Power-Law) Transformation

Executes power-law adjustments to calibrate contrast and dynamic range using a selection of gamma parameters.

1. এই প্রোগ্রামটির মূল Concept কী?

এই প্রোগ্রামটির মূল কাজ হলো একটি Image-এর Brightness পরিবর্তন করা Gamma Transformation ব্যবহার করে।

Gamma Transformation হলো একটি Non-Linear Image Processing Technique, যার মাধ্যমে ছবির Pixel Intensity পরিবর্তন করে ছবিকে উজ্জ্বল (Bright) অথবা অন্ধকার (Dark) করা যায়।

এই প্রোগ্রামে আমরা তিনটি আলাদা Gamma Value ব্যবহার করছি:

gamma = [0.5, 1.0, 2.0]

প্রতিটি Gamma Value দিয়ে Original Image থেকে একটি নতুন Image তৈরি হবে।

তিনটি Gamma Value-এর কাজ:

- Gamma = 0.5: ছবিকে তুলনামূলক বেশি উজ্জ্বল করবে।
- Gamma = 1.0: ছবির কোনো পরিবর্তন করবে না।
- Gamma = 2.0: ছবিকে তুলনামূলক অন্ধকার করবে।

অর্থাৎ, একটি Original Image থেকে আমরা তিনটি আলাদা Gamma Transformed Image তৈরি করব।

2. Mathematical Formula

Gamma Transformation-এর Formula হলো:

$$s = 255 \times \left(\frac{r}{255}\right)^{\gamma}$$

এখানে:

- s = New Pixel Intensity
- r = Original Pixel Intensity
- γ= Gamma Value
- 255 = Maximum Intensity Value

উদাহরণ:

ধরি, Original Pixel Intensity = 100।

এখন তিনটি Gamma Value ব্যবহার করি।

Gamma = 0.5

$$s = 255 \times (100/255)^{0.5}$$

Result ≈ 159 (Brighter)

Gamma = 1.0

$$s = 255 \times (100/255)^1$$

Result = 100 (Unchanged)

Gamma = 2.0

$$s = 255 \times (100/255)^2$$

Result ≈ 39 (Darker)

মনে রাখবে:

- Gamma কম হলে Image উজ্জ্বল হয়।
- Gamma 1 হলে Image অপরিবর্তিত থাকে।
- Gamma বেশি হলে Image অন্ধকার হয়।

3. সম্পূর্ণ Code (সংশোধিত এবং বাংলা Comment সহ)

```
from PIL import Image

image = Image.open("PNG/Task_04_Gamma_Transformation.png")

if image.mode not in ("L", "RGB"):
    image = image.convert("RGB")

width, height = image.size

gamma = [0.5, 1.0, 2.0]

for g in gamma:
    if g <= 0:
        raise ValueError("Gamma value must be greater than 0")

    output_image = Image.new(image.mode, (width, height))

    input_pixels = image.load()
    output_pixels = output_image.load()

    for y in range(height):
        for x in range(width):
```

```
            pixel = input_pixels[x, y]
            if image.mode == "L":
                new_pixel = int(255 * (pixel / 255) ** g)
                output_pixels[x, y] = new_pixel
            else:
                r, g, b = pixel
                new_r = int(255 * (r / 255) ** g)
                new_g = int(255 * (g / 255) ** g)
                new_b = int(255 * (b / 255) ** g)
                output_pixels[x, y] = (new_r, new_g, new_b)
    output_image.save(f"Output/Task_04_Gamma_Transformation_gamma_{g}.png")
    print(f"Gamma transformed image saved as
'Output/Task_04_Gamma_Transformation_gamma_{g}.png'")
```

4. Step-by-Step Code Explanation

Step 1: Import Library

from PIL import Image

এখানে Python-এর Pillow Library থেকে Image Module Import করেছি।

এই Library ব্যবহার করে Image Open, Create, Process এবং Save করতে পারি।

Step 2: Open the Image

image = Image.open("PNG/Task_04_Gamma_Transformation.png")

এই লাইনে নির্দিষ্ট Location থেকে Original Image Open করছি।

Step 3: Check Image Mode

if image.mode not in ("L", "RGB"): image = image.convert("RGB")

এখানে আমরা Image-এর Mode পরীক্ষা করছি।

"L" মানে Grayscale Image এবং "RGB" মানে Color Image।

যদি Image এই দুটি Mode-এর কোনোটিতে না থাকে, তাহলে সেটিকে RGB Format-এ Convert করছি।

এর ফলে আমাদের প্রোগ্রামটি Grayscale এবং RGB দুই ধরনের Image নিয়েই কাজ করতে পারবে।

Step 4: Get Image Dimensions

width, height = image.size

এখানে Original Image-এর Width এবং Height বের করছি।

এই Dimension ব্যবহার করে পরবর্তীতে নতুন Output Image তৈরি করা হবে।

Step 5: Define Gamma Values

gamma_values = [0.5, 1.0, 2.0]

এখানে আমরা তিনটি Gamma Value নির্ধারণ করেছি।

- 0.5 দিয়ে Bright Image তৈরি হবে।
- 1.0 দিয়ে Original Brightness বজায় থাকবে।
- 2.0 দিয়ে Dark Image তৈরি হবে।

একই Image-এর উপর তিনটি আলাদা Gamma Transformation প্রয়োগ করা হবে।

Step 6: Apply Gamma Loop

for gamma_value in gamma_values:

এখানে একটি For Loop ব্যবহার করেছি।

এই Loop তিনবার চলবে, কারণ আমাদের Gamma Value তিনটি।

প্রতিবার Loop চলার সময় একটি Gamma Value নিয়ে সম্পূর্ণ Image Process করা হবে।

এর ফলে তিনটি আলাদা Output Image তৈরি হবে।

Step 7: Validate Gamma Value

if gamma_value <= 0: raise ValueError("Gamma value must be greater than 0")

এখানে Gamma Value পরীক্ষা করছি।

Gamma Value অবশ্যই Zero-এর চেয়ে বড় হতে হবে।

যদি Gamma Value Zero অথবা Negative হয়, তাহলে raise ValueError() ব্যবহার করে Error দেখাবে।

Step 8: Create New Output Image

output_image = Image.new(image.mode, (width, height))

এখানে একটি নতুন Empty Image তৈরি করছি।

নতুন Image-এর Mode এবং Dimension Original Image-এর সমান রাখা হয়েছে।

প্রতিটি Gamma Value-এর জন্য একটি করে নতুন Image তৈরি হবে।

Step 9: Load Pixel Data

input_pixels = image.load()output_pixels = output_image.load()

এখানে দুটি Image-এর Pixel Data Access করছি।

- input_pixels = Original Image-এর Pixel Data।
- output_pixels = নতুন Output Image-এর Pixel Data।

এর মাধ্যমে Original Image থেকে Pixel Value নিয়ে নতুন Image-এ পরিবর্তিত Value বসাতে পারব।

Step 10: Traverse Every Pixel

for y in range(height): for x in range(width):

এখানে দুটি Nested Loop ব্যবহার করেছি।

প্রথম Loop ছবির প্রতিটি Row বরাবর কাজ করে।

দ্বিতীয় Loop প্রতিটি Row-এর প্রতিটি Column-এর Pixel Access করে।

এভাবে ছবির প্রতিটি Pixel একে একে Process করা হয়।

Step 11: Get Original Pixel Value

pixel = input_pixels[x, y]

এখানে Original Image-এর নির্দিষ্ট Coordinate-এর Pixel Value সংগ্রহ করছি।

যদি Image Grayscale হয়, তাহলে একটি Intensity Value পাওয়া যাবে।

আর যদি RGB Image হয়, তাহলে তিনটি Value পাওয়া যাবে: Red, Green এবং Blue।

Step 12: Apply Gamma on Grayscale Image

if image.mode == "L": new_pixel = int(255 * (pixel / 255) ** gamma_value)
output_pixels[x, y] = new_pixel

এখানে প্রথমে পরীক্ষা করছি Image-টি Grayscale কি না।

যদি Grayscale হয়, তাহলে প্রতিটি Pixel-এর উপর Gamma Transformation Formula প্রয়োগ করছি।

Formula ব্যবহার করে নতুন Intensity Value বের করছি।

এরপর output_pixels[x, y] ব্যবহার করে নতুন Image-এর একই Coordinate-এ সেই Value বসিয়ে দিচ্ছি।

Step 13: Apply Gamma on RGB Image

else: red, green, blue = pixel new_r = int(255 * (red / 255) ** gamma_value) new_g = int(255 * (green / 255) ** gamma_value) new_b = int(255 * (blue / 255) ** gamma_value)
output_pixels[x, y] = (new_r, new_g, new_b)

যদি Image RGB হয়, তাহলে প্রতিটি Pixel থেকে তিনটি Color Value সংগ্রহ করছি।

এখানে:

- red = Red Channel Value
- green = Green Channel Value
- blue = Blue Channel Value

এরপর প্রতিটি Channel-এর উপর আলাদাভাবে একই Gamma Transformation Formula প্রয়োগ করছি।

নতুন Red, Green এবং Blue Value বের করে একই Pixel Position-এ বসিয়ে দিচ্ছি।

Step 14: Save Output Image

output_image.save(
f"Output/Task_04_Gamma_Transformation_gamma_{gamma_value}.png")

প্রতিবার Gamma Transformation শেষ হলে নতুন Image-টি Output Folder-এ Save করছি।

এখানে f-string ব্যবহার করা হয়েছে, যাতে File Name-এর মধ্যে Gamma Value স্বয়ংক্রিয়ভাবে যুক্ত হয়।

যেমন:

- ...gamma_0.5.png
- ...gamma_1.0.png
- ...gamma_2.0.png

সবশেষে:

print(f"Gamma transformed image saved for gamma = {gamma_value}")

প্রতিটি Image Save হওয়ার পর একটি Confirmation Message দেখাবে।

Oral Explanation (বাংলা)

প্রথমে আমি Pillow Library Import করেছি, যেটি Python-এ Image Processing-এর জন্য ব্যবহার করা হয়।

এরপর একটি Original Image Open করেছি। তারপর Image-এর Mode পরীক্ষা করেছি, যাতে Image-টি Grayscale অথবা RGB Format-এ থাকে। অন্য কোনো Format হলে সেটিকে RGB-তে Convert করেছি।

এরপর Image-এর Width এবং Height বের করেছি।

তারপর তিনটি Gamma Value নিয়েছি, যথাক্রমে 0.5, 1.0 এবং 2.0। এই Gamma Value ব্যবহার করে আমরা তিনটি আলাদা Output Image তৈরি করব।

এরপর একটি For Loop ব্যবহার করেছি, যা প্রতিটি Gamma Value-এর জন্য কাজ করবে।

প্রথমে Gamma Value Positive কি না তা পরীক্ষা করেছি। তারপর Original Image-এর সমান Dimension ব্যবহার করে একটি নতুন Output Image তৈরি করেছি।

এরপর load() Function ব্যবহার করে Input এবং Output Image-এর Pixel Data Access করেছি।

তারপর দুটি Nested Loop ব্যবহার করে প্রতিটি Pixel Access করেছি।

যদি Image Grayscale হয়, তাহলে সরাসরি Gamma Transformation Formula প্রয়োগ করেছি।

আর যদি Image RGB হয়, তাহলে Red, Green এবং Blue Channel আলাদাভাবে সংগ্রহ করে প্রতিটি Channel-এর উপর Gamma Transformation Formula প্রয়োগ করেছি।

Formula-টি হলো 255 multiplied by Original Pixel divided by 255, raised to the power of Gamma Value।

এরপর নতুন Pixel Value Output Image-এ বসিয়েছি।

সব Pixel Process করা শেষ হলে Output Image Save করেছি।

এভাবে তিনটি Gamma Value ব্যবহার করে তিনটি আলাদা Brightness-এর Image তৈরি করেছি।

6. Viva Questions & Answers

Q1. What is Gamma Transformation?

Ans: Gamma Transformation is a nonlinear image processing technique used to adjust the brightness of an image.

Q2. What is the Gamma Transformation Formula?

Ans:

$$s = 255 \times (r/255)^{\gamma}$$

Q3. What happens when Gamma is less than 1?

Ans: The image becomes brighter.

Q4. What happens when Gamma is equal to 1?

Ans: The original image remains unchanged.

Q5. What happens when Gamma is greater than 1?

Ans: The image becomes darker.

Q6. Why do we use 255 in the formula?

Ans: Because 255 is the maximum intensity value of an 8-bit image.

Q7. Why do we divide the pixel value by 255?

Ans: To normalize the pixel intensity into a range between 0 and 1.

Q8. Why do we use image.mode?

Ans: To identify whether the image is Grayscale or RGB.

Q9. Why do we use Image.new()?

Ans: To create a new output image for each gamma value.

Q10. Why do we use Nested Loops?

Ans: To access and process every pixel in the image.

Q11. Can Gamma Transformation work on RGB images?

Ans: Yes. We can apply gamma transformation separately to the Red, Green and Blue channels.

Q12. What is the difference between Gamma Transformation and Negative Transformation?

Ans: Negative Transformation reverses pixel intensity using 255 - pixel, whereas Gamma Transformation adjusts pixel intensity using a power-law formula.

7. গুরুত্বপূর্ণ বিষয়গুলো মনে রাখবে

Code / Function	কাজ
Image.open()	Image Open
image.mode	Image Mode Check
image.convert()	Image Format Convert
image.size	Width ও Height
gamma_values	Gamma Value সংরক্ষণ
Image.new()	নতুন Image তৈরি
image.load()	Pixel Access
for loop	প্রতিটি Pixel Process
** gamma_value	Power Operation
output_pixels[x,y]	Pixel Update
image.save()	Image Save